

Funzioni ricorsive

Funzioni ricorsive

Funzioni che risolvono un problema chiamando se stesse.

Cos'è la ricorsione?

Una funzione ricorsiva è una funzione che, per risolvere un problema, **chiama se stessa** con una versione più piccola dello stesso problema.

Un'analogia: immagina di dover smontare una bambola matrioska (quelle russe una dentro l'altra). Come la apri? Apri la prima, trovi un'altra matrioska più piccola, la apri, trovi un'altra ancora più piccola... finché non trovi quella tinyissima che non si apre più. Quella è il **caso base**.

Le due parti fondamentali

Ogni funzione ricorsiva deve avere:

1. **Il caso base** — la condizione che fa fermare la ricorsione
2. **Il caso ricorsivo** — dove la funzione chiama se stessa con un problema più piccolo

Senza il caso base, la funzione si chiamerebbe all'infinito e il programma si bloccherebbe con un errore.

Esempio classico: il fattoriale

Il fattoriale di un numero (scritto con il simbolo **!**) è il prodotto di tutti i numeri interi da 1 fino a quel numero:

$$\begin{aligned}5! &= 5 \times 4 \times 3 \times 2 \times 1 = 120 \\3! &= 3 \times 2 \times 1 = 6 \\1! &= 1\end{aligned}$$

Si può anche scrivere così: $5! = 5 \times 4!$. E $4! = 4 \times 3!$. Vedi il pattern? Il problema si riduce sempre.

```
def fattoriale(n):
    # Caso base: se n è 0 o 1, il fattoriale è 1
    if n == 0 or n == 1:
        return 1
    # Caso ricorsivo: n * fattoriale del numero precedente
    return n * fattoriale(n - 1)

print(fattoriale(5))  # 120
print(fattoriale(3))  # 6
```

Ecco cosa succede passo per passo quando chiami `fattoriale(5)` :

```
fattoriale(5)
  → 5 × fattoriale(4)
    → 4 × fattoriale(3)
      → 3 × fattoriale(2)
        → 2 × fattoriale(1)
          → 1 ← caso base, si ferma!
```

Poi si torna indietro calcolando: `2×1=2` , `3×2=6` , `4×6=24` , `5×24=120` .

Esempio: la successione di Fibonacci

La successione di Fibonacci è famosa in matematica e in natura: `0, 1, 1, 2, 3, 5, 8, 13, 21, ...`

Ogni numero è la somma dei due che lo precedono. Anche questo si presta alla ricorsione:

```
def fibonacci(n):
    if n == 0:
        return 0
    if n == 1:
        return 1
    return fibonacci(n - 1) + fibonacci(n - 2)

for i in range(8):
    print(fibonacci(i), end=" ")
# 0 1 1 2 3 5 8 13
```

Un limite importante

Python può gestire al massimo circa 1000 chiamate ricorsive in profondità. Se superi questo limite, ottieni un `RecursionError`. Per questo la ricorsione non è sempre la scelta migliore per problemi con numeri molto grandi.

Quando usare la ricorsione?

La ricorsione è utile quando il problema ha una struttura naturalmente "annidata" — cioè quando si può descrivere un problema grande come una versione più piccola dello stesso problema.

Per problemi semplici (come contare fino a 100, o scorrere una lista), usa i cicli `for` o `while`: sono più veloci e più facili da capire.